Data Structures and Algorithm DSA

Lab Project

Smart Parking System with Analytics

DSA-SE-Semester Project

Name 1: Kashif Idrees (L1F23BSSE0172)

Name 2: Muhammad Umar (L1F23BSSE0137)

Name 3: Rana Subhan Iqbal (L1F23BSSE0169)

Section: P3

Instructor: Sir Fitrus Abbas

Date: 25-June-2025

Comprehensive Design Analysis:

The **Smart Parking System** is a Java Swing GUI application that manages parking slots for three types of vehicles: **Car**, **Bike**, and **Truck**. The design is modular and based on a clean **Model-View-Controller** approach using Java Swing components for the interface and internal logic for control.

Main Functionalities:

- Park a vehicle by type and number.
- Show available/reserved/occupied slots in a grid.
- Reserve or cancel reservations for specific slots.
- Check out a parked vehicle.
- View the current parking status.

UI Components Used:

- JFrame, JPanel, JButton, JLabel, JTextField, JComboBox, JTextArea
- Layouts: FlowLayout, BorderLayout, GridLayout

The application maintains real-time status updates for each slot using color-coding:

Green – Available
Red – Occupied
Blue – Reserved

2. Algorithm Details with Complexity Analysis

a. Slot Selection & Vehicle Parking

- **Algorithm**:

- On vehicle entry, display all available slots.
- User selects an empty slot to park
- Slot state is updated, and the vehicle is recorded.

- **Time Complexity**:
  - Lookup in HashMap: **O (1)**
  - Traversing slot matrix: **O(n²)** for each type, where n = rows or cols (4x4 in this case)

b. Check-out Operation

- **Algorithm**:
  - Get vehicle number from input.
  - Use HashMap to find position.
  - Update slot matrix to set the value to 0 (empty).
- **Time Complexity**:
  - **O (1)** for vehicle lookup
  - **O (1)** for matrix update

c. Reservation / Cancellation

- **Algorithm**:
  - Toggle slot value to 2 (reserve) or back to 0 (cancel) depending on mode.
  - Button interactions update UI accordingly.
- **Time Complexity**:
  - Reservation scan: **O(n²)** per vehicle type

d. Display Status

- **Algorithm**:

  Iterate through HashMap to display parked vehicles.

  Traverse each slot matrix to find reserved slots.

- **Time Complexity**:

O(k) for parked vehicles (k = number of vehicles)

O(n^2) for each matrix (3 matrices total)

3. Data Structure Selection Justification:

Data Structure	Use Case	Reason for Selection
HashMap<String, String>	Vehicle number → slot mapping	Fast lookups for parking/check-out operations.
int [][] (3 arrays)	Parking slot grid for each type	Simple grid structure, easy to traverse and modify.
JComboBox, JTextField, JButton	GUI Components	Standard Swing elements for dynamic interactivity.

Justification:

- HashMap gives **constant-time complexity** for search, insert, and delete – ideal for real-time vehicle tracking.
- int[][] allows intuitive representation of slots, where 0 = empty, 1 = occupied, and 2 = reserved. This makes visual representation and logic toggling simple and effective.

4. Sample Analytics & Discussion:

a. Slot Utilization Rate

Assuming:

- Car slots: 4x4 = 16
- Bike slots: 4x4 = 16
- Truck slots: 4x4 = 16

Total: 48 slots.

If:

- 10 vehicles parked
- 5 slots reserved

Then:

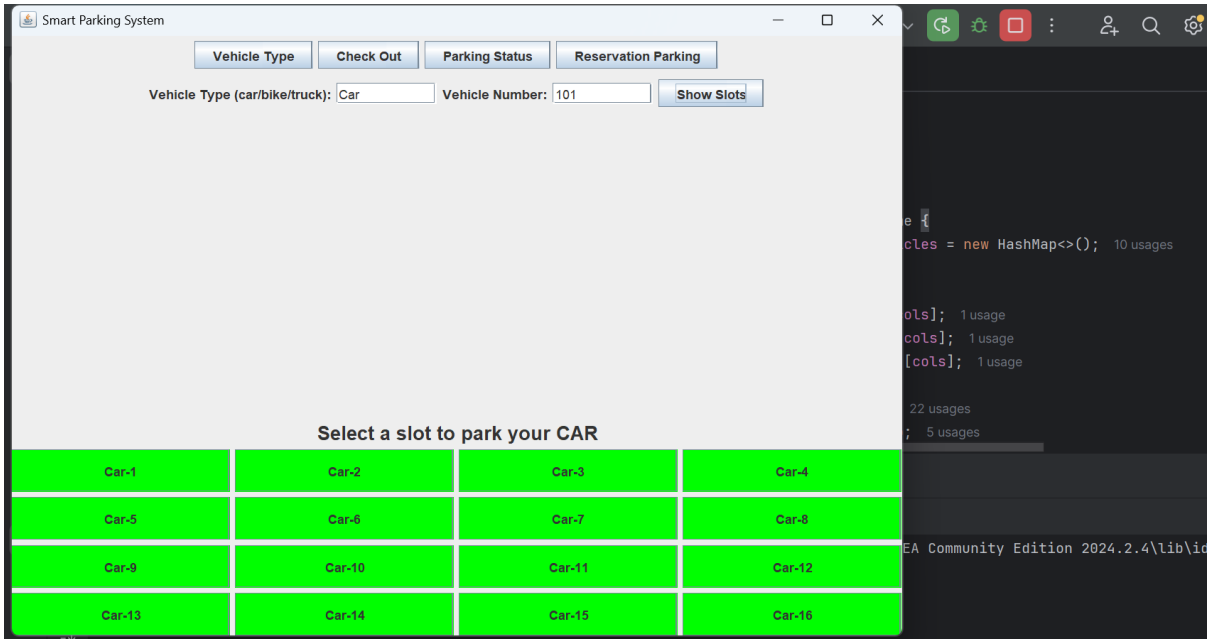
- Utilization = (10 / 48) * 100 = **20.83%**
- Reserved = (5 / 48) * 100 = **10.41%**

b. Scalability Consideration

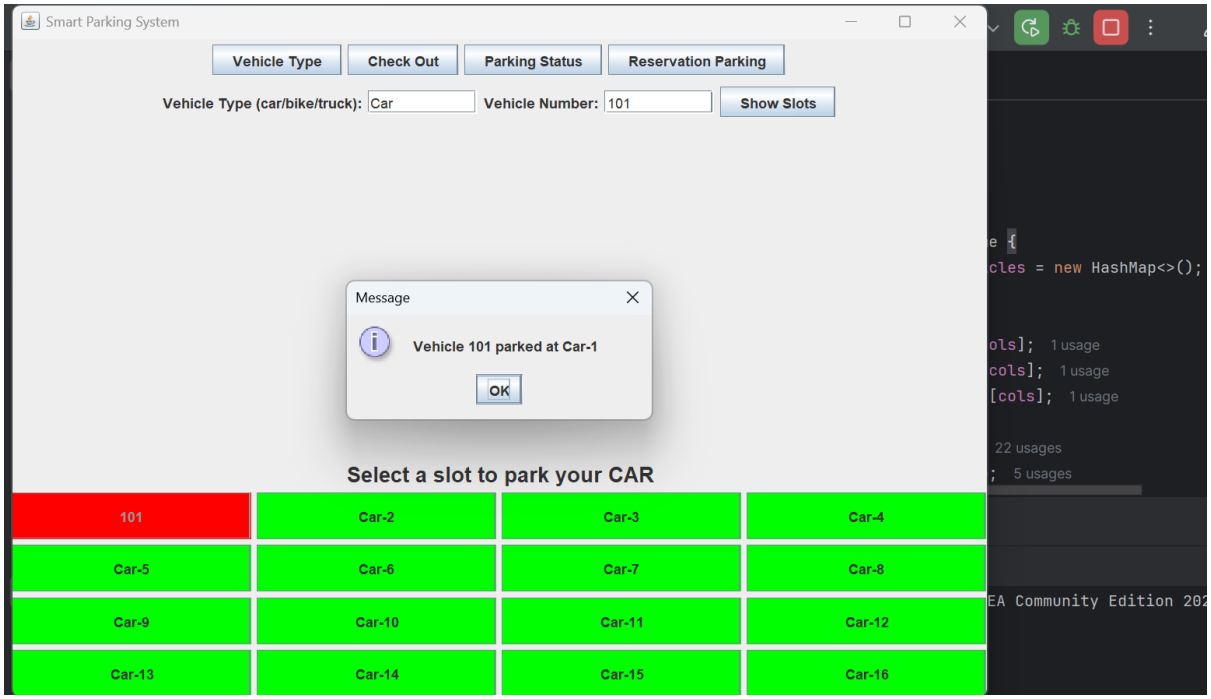
- The system can be scaled by increasing rows and cols.
- For larger lots, a dynamic or database-backed solution is recommended.
- Slot states could be encapsulated in objects instead of plain int values for richer data (e.g., timestamp, duration).

Output:

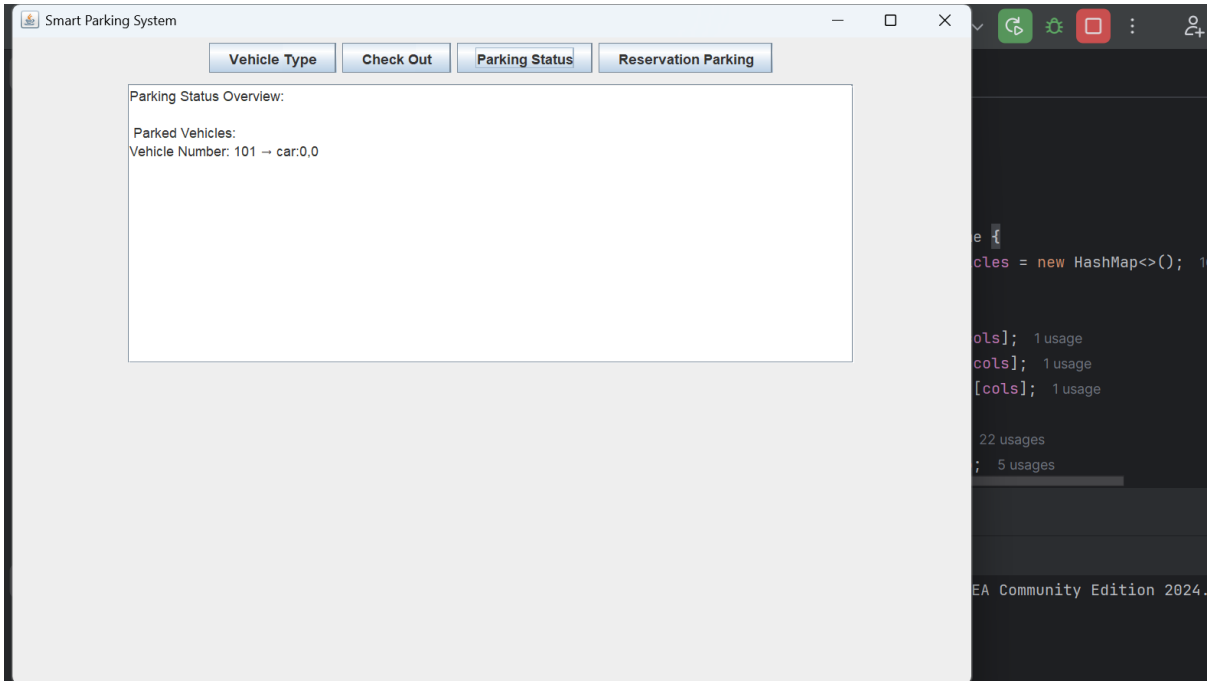
1. Type Car/Bike/Truck, Type vehicle number and click to slots, then they show car slots.



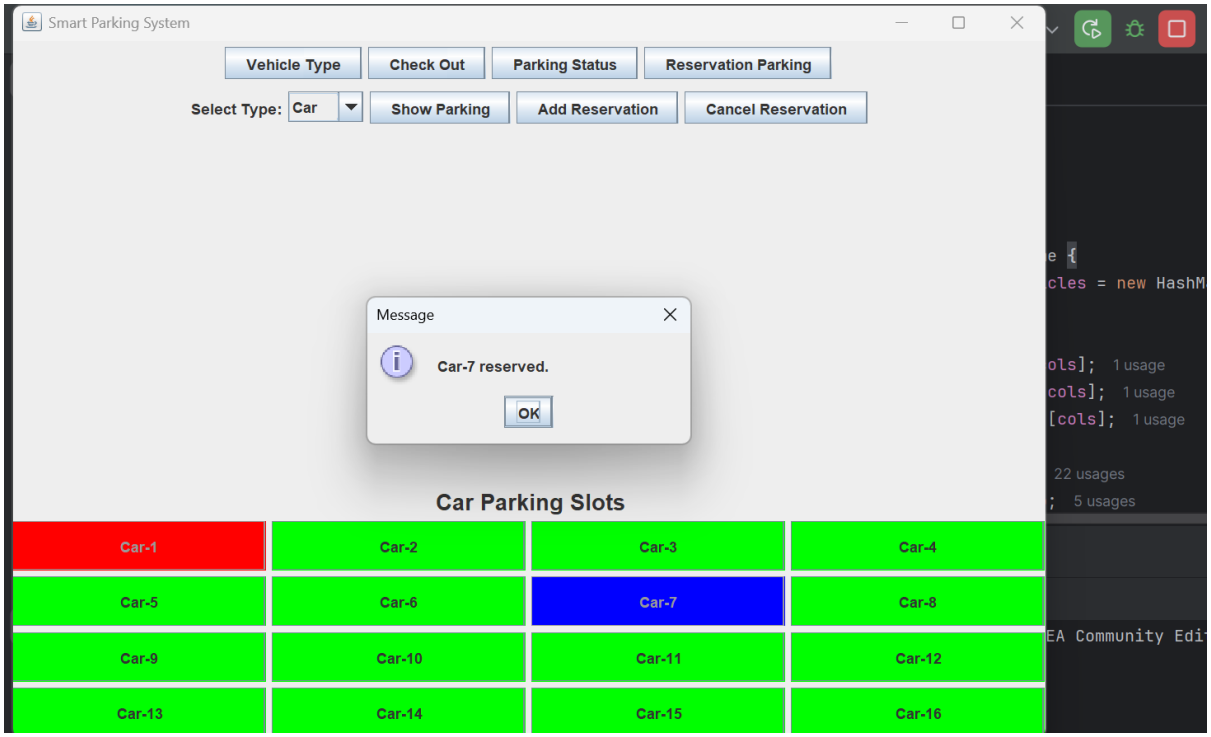
2. Then when we select a slot, they deliver the message to user.



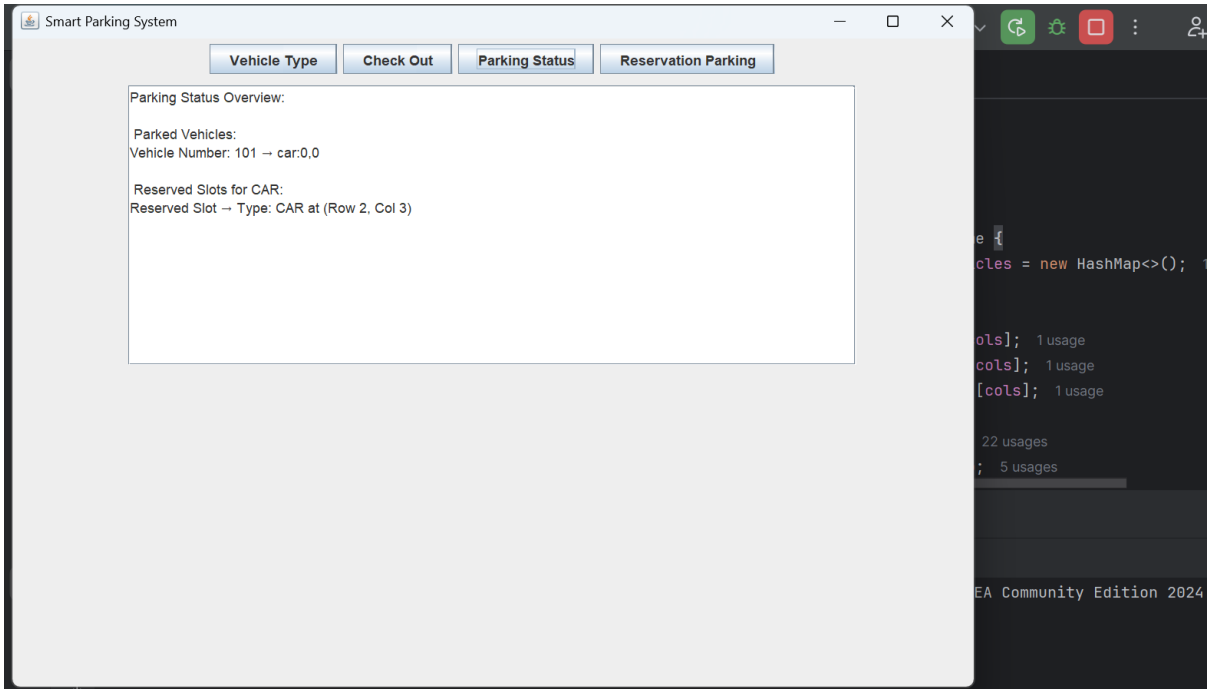
3. User overview the parking status by click on Parking Status Bar.



4. Then, when we add more than one reservation by click the Add reservation Car will be reserved.



5. We also overview the parking status at reserved or not.



Conclusion:

The Smart Parking System efficiently simulates parking operations using basic data structures with a responsive Swing UI. With optimizations and modularity, it can evolve into a full-scale management application.